streaming the response to the client.

We've finished simplifying the `RecipeApiController` now. By extracting various pieces of functionality to filters, the original controller in listing 22.1 has been simplified to the version in listing 22.2. This is obviously a somewhat extreme and contrived demonstration, and I'm not advocating that filters should always be your go-to option.

**TIP** Filters should be a last resort in most cases. Where possible, it is often preferable to use a simple private method in a controller, or to push functionality into the domain instead of using filters. Filters should generally be used to extract repetitive, HTTP-related, or common cross-cutting code from your controllers.

There's still one more filter we haven't looked at yet, because it applies only to Razor Pages: page filters.

### *22.1.6 Page filters: Customizing model binding for Razor Pages*

As already discussed, action filters apply only to controllers and actions; they have no effect on Razor Pages. Similarly, page filters have no effect on controllers and actions. Nevertheless, page filters and action filters fulfill similar roles.

As is the case for action filters, the ASP.NET Core framework includes several page filters out of the box. One of these is the Razor Page equivalent of the caching action filter, `ResponseCacheFilter`, called `PageResponseCacheFilter`. This works identically to the action-filter equivalent I described in section 22.1.3, setting HTTP caching headers on your Razor Page responses.